UNIVERSITÉ DU QUÉBEC EN OUTAOUAIS

COURS : Conception des systèmes digitaux

SIGLE DU COURS : GEN1243

PROJET DE COURS :

« Contrôle automatique des barrières de sécurité et des feux de signalisation du passage à niveau pour trains et voitures. »

Professeur : M. Rahmani Naim Mohamed

SESSION : AUTOMNE 2024

PAR :

Tahoc Wokam Durrice Léa	**TAHD63260502**
Soma Marion Laurence Malika	**SOMM17599900**
Mbiandou Mbiadjeu Morel Bonheur	**MBIM83310509**

REMERCIEMENTS

Nous ne saurions entamer ce projet sans exprimer notre profonde gratitude à l'Éternel Tout-Puissant, pour la force et la détermination qu'Il nous a accordées pour mener à bien ce travail.

Nous souhaitons également adresser nos sincères remerciements à toutes les personnes ayant contribué, de près ou de loin, à la réalisation de ce projet ainsi qu'à la rédaction de ce rapport.

Tout d'abord, nos remerciements les plus chaleureux vont à **M. Rahmani Naim Mohamed**, notre professeur, pour son encadrement précieux et ses conseils avisés tout au long de ce projet. Sa patience, son soutien et son expertise ont été des atouts essentiels à notre progression.

Un immense merci à **M. Ali Abdallah Guire**, responsable des laboratoires, pour avoir mis à notre disposition un environnement de travail propice et pour son soutien constant tout au long de cette aventure.

Nous exprimons également notre gratitude envers **M. Chebihi Abdelkrim**, technicien en travaux pratiques, pour son assistance technique précieuse et son aide indéfectible.

Enfin, nous tenons à remercier l'ensemble du personnel et des étudiants de l'institut, dont l'engagement a contribué à créer un cadre stimulant, propice à l'apprentissage et à l'innovation. Votre passion et votre dévouement nous inspirent au quotidien.

Ce projet a été une expérience d'apprentissage exceptionnelle, et nous sommes profondément reconnaissants pour toutes les opportunités et le soutien reçus.

Merci à tous.

Résumé

Ce rapport présente la conception d'un système de contrôle automatisé pour un passage à niveau ferroviaire, utilisant un FPGA afin de gérer la détection des trains, les signaux lumineux, les barrières de sécurité et les avertissements sonores. Le but principal de ce système est de prévenir les accidents en assurant une protection efficace pour les usagers de la route et les piétons lors du passage d'un train. Grâce à une redondance des capteurs, le système détecte avec précision les trains en approche, ainsi que la présence d'objets ou de personnes sur les voies. En cas d'obstacle détecté, un signal d'alerte est envoyé au conducteur du train pour l'avertir d'un danger. La gestion automatique de la barrière permet une régulation fluide du trafic en minimisant les délais de fermeture. Ce projet intègre également une option de contrôle manuel durant une certaine plage horaire (de 18h à 8h), accessible par code PIN pour garantir un fonctionnement adapté aux besoins opérationnels.

Terminologie

1. FPGA : Circuits intégrés programmables utilisés pour exécuter des tâches en temps réel avec une grande flexibilité et efficacité.
2. Passage à niveau : Croisement entre une route et une voie ferrée, nécessitant des dispositifs pour sécuriser le trafic.
3. Automatisation : Processus visant à remplacer les interventions humaines par des systèmes automatiques.
4. Sécurité : Ensemble des mesures prises pour protéger les usagers et prévenir les accidents.
5. Détection de présence : Utilisation de capteurs pour identifier la présence d'objets, véhicules ou piétons dans une zone spécifique.
6. Signalisation : Système de feux et messages d'alerte pour informer et diriger les usagers.
7. Transport intelligent : Technologies intégrées pour améliorer la gestion et la sécurité des infrastructures de transport.
8. Système anticollision : Mécanisme conçu pour éviter les collisions entre le train et des obstacles.
9. Redondance des capteurs : Utilisation de plusieurs capteurs pour assurer la fiabilité en cas de panne.
10. Contrôle d'accès : Régulation de l'accès aux zones sensibles, comme les voies ferrées, à l'aide de barrières et signaux.

Table des matières

REMERCIEMENTS 1
I. Liste des figures 4
I. Introduction 5
II. Spécification du système 5
III. Description et Justification de la solution matérielle et logicielle 6
a. Outils et environnement de développement 6
b. Matériel de prototypage : 7
IV. Schéma Global du Système 10
V. Architecture du système 13
VI. Flowchart et algorithme 16
VII. Développement et validation 18
a. Code VHDL 18
b. Schémas électriques des connexions 24
c. Simulations fonctionnelles 24
d. Représentations Graphiques du Système 26
VIII. Difficultés rencontrées lors de la conception 28
IX. Conclusion et perspectives 29
X. Bibliographie 31
XI. Annexes 31

I. Liste des figures

Figure 1:Carte DE1-SOC Cyclone V 6
Figure 2: Servomoteur - Micro-servo 9g, SG90 7
Figure 3: capteurs Ultrasons 8
Figure 4: LEDs 8
Figure 5: Buzzer passif 9
Figure 6: Connecteurs et fils 9
Figure 7: breadboards 10
Figure 8: schéma globale du système 10
Figure 9: schéma FSM 16
Figure 10: flowchart 17
Figure 11: code affichage, leds, messages et buzzer 20
Figure 12: code feux clignotants 21
Figure 13: code capteurs ultrasons 22
Figure 14: code servomoteur 23
Figure 15: schémas électriques des connexions 24
Figure 16: simulation servomoteurs 25
Figure 17: simulation buzzer 25
Figure 18: simulation messages, leds et code pins 26
Figure : Symbole graphique du système 26
Figure : Netlist du système 27

I. Introduction

Ces dernières années, les FPGAs (Field Programmable Gate Arrays) sont devenus des composants incontournables dans les applications en temps réel, en raison de leur capacité à gérer des tâches simultanées. Leur principal avantage réside dans l'exécution parallèle, limitée uniquement par les ressources matérielles disponibles. Cela en fait une solution idéale pour des systèmes de contrôle automatisés comme la gestion d'une barrière de passage à niveau, où plusieurs processus doivent être effectués en parallèle pour garantir sécurité et réactivité.

Dans ce projet, un FPGA est utilisé pour concevoir un système automatique de contrôle de barrière et de feux de signalisation pour un passage à niveau. L'objectif est de développer un mécanisme anticollision capable de prévenir les accidents en détectant les obstacles proches et en prenant les mesures nécessaires pour assurer la sécurité des usagers de la route. Le contrôle de la barrière et des signaux de sécurité sera géré en VHDL (VHSIC Hardware Description Language), permettant de tirer parti de la logique combinatoire pour réagir rapidement aux changements dans l'environnement.

Ce système peut être utilisé dans divers contextes nécessitant une automatisation et une haute réactivité, comme les systèmes de transport intelligent (V2X), tout en renforçant la sécurité grâce à la détection précise des objets à proximité. Bien que des microprocesseurs puissent également être utilisés pour de tels systèmes, ils sont limités en matière de traitement parallèle, ce qui fait des FPGAs une solution optimale pour cette application critique.

II. Spécification du système

Les objectifs spécifiques de ce système sont les suivants :

1. **Détection et abaissement de la barrière** : Lorsqu'un train s'approche, le système détecte sa présence via un capteur dédié et abaisse automatiquement la barrière pour empêcher tout véhicule ou piéton de traverser les voies pendant le passage du train.
2. **Signalisation visuelle et sonore** : Une lumière verte est activée pour le conducteur du train, autorisant son passage, tandis qu'un signal rouge est destiné aux véhicules et piétons. Un message « STOP » s'affiche sur les afficheurs à 7 segments. En parallèle, un signal sonore informe les usagers de la route de la présence d'un train en transit.
3. **Réouverture automatique de la barrière** : La barrière se relève dès que le train a terminé de traverser, évitant ainsi un encombrement inutile de la circulation près du passage.
4. **Automatisation et contrôle manuel** : L'opérateur active ou désactive l'automatisme en saisissant un code PIN via un clavier ou en utilisant des interrupteurs sur la carte FPGA.
5. **Double détection pour redondance** : Par mesure de sécurité, la détection de la présence d'un train repose sur deux dispositifs indépendants, garantissant une redondance et minimisant les risques de défaillance.

6. **Détection de présence sur les voies** : Le système est équipé de capteurs capables de détecter la présence d'un objet, véhicule ou piéton sur les voies. En cas d'obstruction, la progression du train est immédiatement arrêtée.
7. **Signal d'urgence pour le train** : Si jamais, un objet est détecté sur les voies alors qu'un train s'approche ou si la barrière reste ouverte pour une raison quelconque, un signal rouge clignotant et une alarme sonore alertent le conducteur du train, qui doit stopper son véhicule avant le passage à niveau. Le train ne reprend sa route qu'une fois l'obstacle écarté.

III. Description et Justification de la solution matérielle et logicielle

Cette section présente les outils utilisés, ainsi que les choix de conception pour développer et implémenter le système automatique de passage à niveau.

a. Outils et environnement de développement

- Langage VHDL : Utilisé pour modéliser et programmer la logique de contrôle sur le FPGA. Le VHDL permet de concevoir des systèmes numériques en décrivant leur comportement et leur architecture.

- Carte FPGA DE1-SoC Cyclone V : La carte FPGA DE1-SoC, équipée du Cyclone V, est le cœur du système. Elle permet de contrôler les différents périphériques grâce à sa capacité de traitement en parallèle et à sa flexibilité pour la conception matérielle via VHDL.

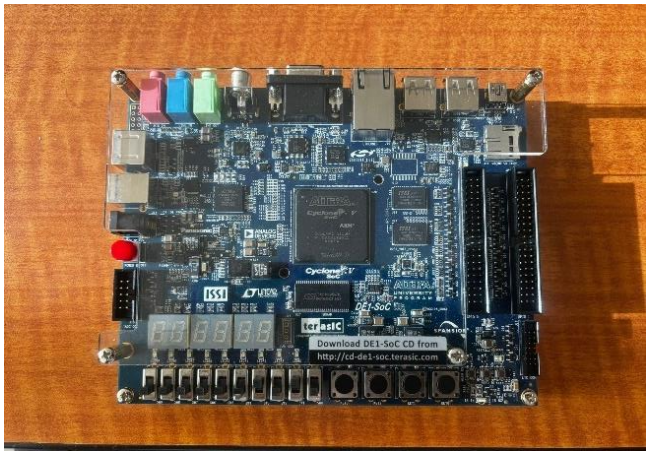


Figure 1:Carte DE1-SOC Cyclone V

- Logiciel Quartus Prime : Utilisé pour la compilation, la simulation et la synthèse des codes VHDL. Il a également permis de configurer les broches de la FPGA pour interfacer les périphériques.

b. Matériel de prototypage :

Dans ce projet, plusieurs composants matériels ont été utilisés pour concevoir et tester le système de passage à niveau automatique. Voici une description détaillée de chaque élément :

- **Servomoteur : Micro-servo 9g, SG90**

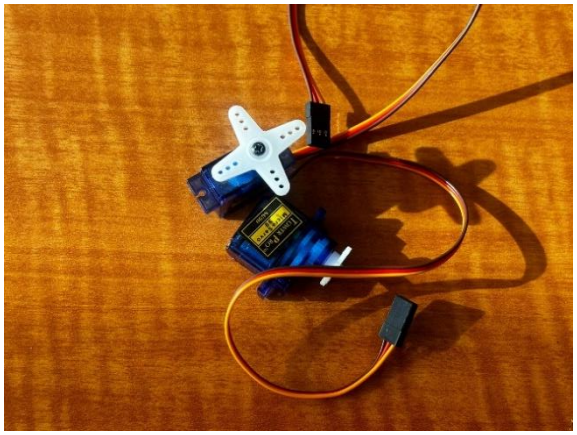


Figure 2: Servomoteur - Micro-servo 9g, SG90

Le micro-servo SG90 est utilisé pour contrôler le mouvement de la barrière :

- **Précision et fiabilité** : Ce servomoteur fournit une rotation contrôlée dans un angle spécifique (généralement 0-180°), essentiel pour ouvrir et fermer la barrière de manière fluide et précise.
- **Faible consommation d'énergie** : Idéal pour un projet nécessitant un matériel compact et peu énergivore.
- **Compatibilité** : Il est facilement pilotable via des signaux PWM générés par le FPGA.

- **Capteurs à ultrasons :**



Figure 3: capteurs Ultrasons

Le capteur HC-SR04 est utilisé pour détecter les trains et les obstacles :

- **Portée et précision** : Ce capteur mesure des distances avec une précision de quelques millimètres, ce qui est crucial pour détecter les trains en approche et les obstacles sur la voie.
- **Rapidité** : Il offre une mesure quasi-instantanée, garantissant une réactivité rapide du système en cas de détection critique.
- **Facilité d'intégration** : Il fonctionne avec des signaux numériques facilement gérables par le FPGA, rendant son interface simple.

Le système utilise 5 capteurs ultrasons :

- **Deux capteurs à l'entrée** et **deux à la sortie** pour détecter la présence ou l'absence du train.
- **Un capteur sur la voie** pour identifier les obstacles.

➢ **LEDs (rouges et vertes)**

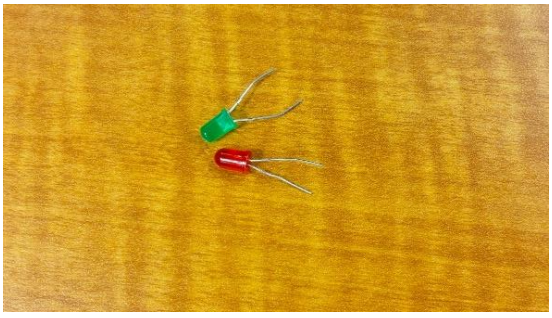


Figure 4: LEDs

Les LEDs servent à indiquer l'état du passage à niveau aux usagers (piétons, conducteurs et train) :

- **LEDs rouges** : Signalent que la barrière est fermée ou que la voie est occupée pour les piétons et qu'un obstacle est présent pour les conducteurs de trains.
- **LEDs vertes** : Indiquent que la voie est libre pour les usagers.
- **Visibilité** : Ces LEDs garantissent une signalisation claire et visible même dans des conditions de faible luminosité.

➢ **Buzzer passif**

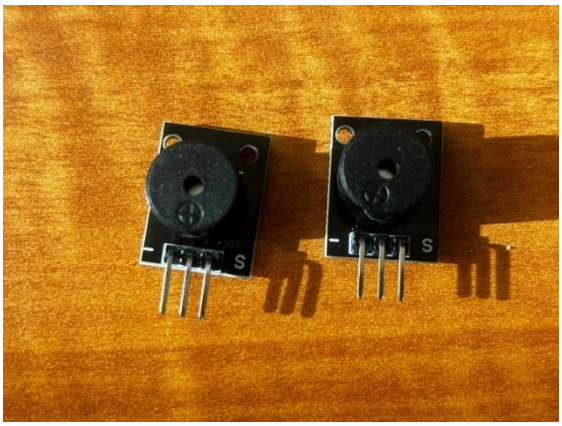


Figure 5: Buzzer passif

Le buzzer KY-006 est utilisé pour fournir des alertes sonores en cas de danger ou de passage imminent du train :

- **Simplicité** : Ce composant peut produire des sons à différentes fréquences, ce qui permet de varier les signaux sonores en fonction des situations (par exemple, alarme ou avertissement).
- **Interaction facile avec le FPGA** : Le buzzer est contrôlé via une sortie PWM du FPGA, ce qui permet une modulation simple du son.

➢ **Connecteurs et fils**

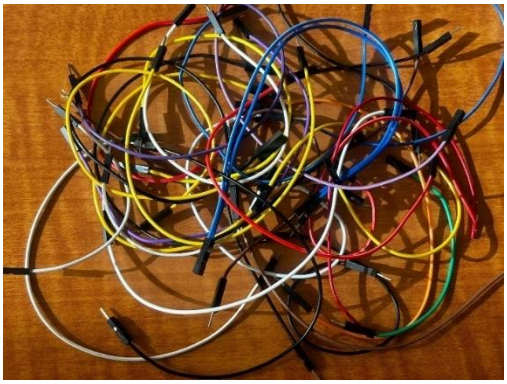


Figure 6: Connecteurs et fils

Des connecteurs et fils électriques sont utilisés pour relier les capteurs, LEDs, servomoteurs, et autres composants à la carte FPGA et à la breadboard.

- **Breadboards**

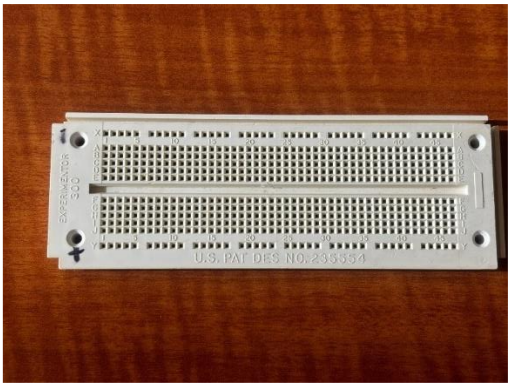


Figure 7: breadboards

Permettent un prototypage rapide et flexible pour tester les connexions.

IV. Schéma Global du Système

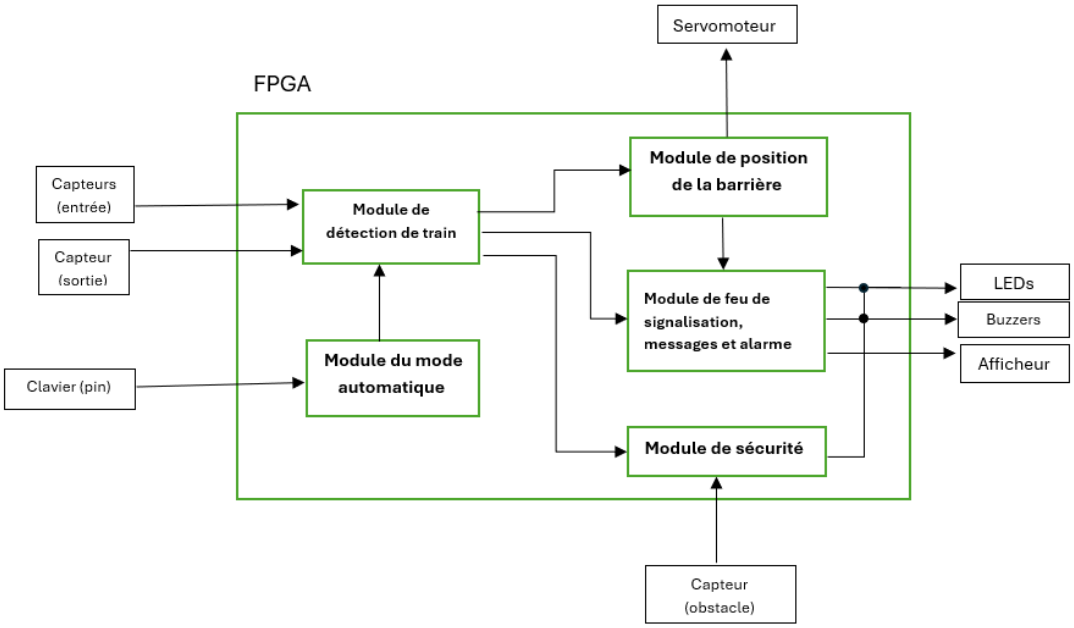


Figure 8: schéma globale du système

❖ *Module de détection de train*

- **Rôle** : Identifier la présence d'un train grâce à 4 capteurs (2 d'entrée et 2 de sortie en redondance).
- **Entrées** : Signaux des capteurs capteur_1 et capteur_2 (entrée), capteur_3 et capteur_4 (sortie).
- **Sorties** :
  - **Train**: Indique si un train est dans la zone.
- **Fonctionnement** :
  - Si deux capteurs (entrée ou sortie) détectent un objet à < 13.75 cm, le train est signalé.
  - La redondance assure la fiabilité, et une incohérence déclenche une alerte.
- **Interactions** : Contrôle la barrière, active le mode automatique et alerte en cas d'erreur de capteur.

❖ *Module de position de la barrière*

- **Rôle** : Contrôler la position de la barrière en fonction de la présence d'un train et d'un obstacle.
- **Entrées** :
  - **Train** : Signal provenant du module de détection (présence du train).
  - **Obstacle** : Signal d'un capteur vérifiant la présence d'un obstacle sur la voie.
  - **Mode auto** : signal provenant du module automatique (mode automatique ou manuelle)
- **Sorties** :
  - **Barriere** : Indique si la barrière est ouverte ou fermée.
- **Fonctionnement** :
  - La barrière se ferme uniquement si un train est détecté (**Train = 1**) et qu'aucun obstacle n'est présent (**Obstacle = 0**).
  - Si un obstacle est détecté, la barrière reste ouverte et un signal peut être envoyé au module d'alarme.
- **Interactions** :
  - Reçoit le signal de détection du train et de l'obstacle.
  - Envoie la position de la barrière au module de sécurité

❖ *Module de feu de signalisation, messages et alarme*

- **Rôle** : Gérer les feux de signalisation, les messages et les alarmes pour les utilisateurs et le train, en fonction de l'état de la barrière, de la présence d'un train ou d'un obstacle.

- **Entrées** :

- **Barrière** : Position actuelle de la barrière (ouverte/fermée).
- **Train** : Signal du module de détection (présence du train).
- **Obstacle** : Signal du capteur d'obstacle.

- **Sorties** :
  - **User_Traffic_Light** : Feu rouge/vert pour les utilisateurs.
  - **Train_Traffic_Light** : Feu rouge/rouge clignotant pour le train.
  - **Message_Display** : Messages **"STOP"** ou **"RAS"** pour les utilisateurs.
  - **Alarm_Sound** : Deux types d'alarme (utilisateur/train).

- **Fonctionnement** :
  - **Situation normale (pas d'obstacle)** :
    - **Barrière fermée** :
      - Feu rouge pour les utilisateurs, affiche **"STOP"**, alarme utilisateur active.
      - Feu vert pour le train.
    - **Barrière ouverte** :
      - Feu vert pour les utilisateurs, affiche **"RAS"**.
      - Feu rouge pour le train.
  - **Obstacle détecté** :
    - Feu rouge pour les utilisateurs avec message **"STOP"**.
    - Feu rouge clignotant pour le train, alarme train active pour signaler un danger.

- **Interactions** :
  - Reçoit les signaux des modules de détection et de barrière.
  - Envoie les signaux de feu, message et alarme au module de sécurité.

❖ ***Module du mode automatique***

- **Rôle** : Bascule entre les modes automatique et manuel via la validation d'un code PIN.
- **Entrées** :
  - **PIN_Code** : Code saisi par l'utilisateur.
- **Sorties** :
  - **Current_Mode** : Mode_auto (0 = manuel, 1 = automatique).
- **Fonctionnement** :
  - Si le code est validé, le mode bascule selon la demande. Sinon, le mode reste inchangé.
- **Interactions** : Active ou désactive les fonctions automatiques des autres modules.

❖ ***Module de sécurité***

- **Rôle** : Garantir la sécurité en cas de détection d'un obstacle ou d'une incohérence dans le système.
- **Entrées** :
  - **Obstacle**: Signal d'obstacle sur la voie.
  - **Train**: Présence d'un train.
  - **Barriere**: Position actuelle de la barrière.
- **Sorties** :
  - **Train_Light_Blinking** : Feu rouge clignotant pour alerter le train.
  - **Train_Alarm** : Son d'alerte spécifique pour le train.
- **Fonctionnement** :
  - Si un obstacle est détecté et qu'un train est présent ou que la barrière est fermée de manière incohérente, le feu rouge clignote et l'alarme du train s'active.
- **Interactions** : Fonctionne en coordination avec les modules de détection, signalisation et barrière.

V. Architecture du système

a. FSM (Finite State Machine) - Machine à États Finis

La FSM permet de définir les différents **états** du système et les **transitions** entre eux en fonction des conditions.

- **Table d'états**

État Actuel	Condition/ Évènement	État Suivant	Action
Idle (mode manuel)	Le code PIN correct est entré	Mode automatique	-Feux verts pour véhicules. - Message "RAS".
Mode automatique	Un train est détecté	Train Approchant	- Feux rouges pour véhicules - Message "STOP" - Préparer la barrière.
Train Approchant	Le train est proche	Fermeture Barrière	- Début de la fermeture de la barrière.

Fermeture Barrière	La barrière est complètement fermée	Train en Passage	- Barrière fermée. - Signal sonore pour passage du train.
Train en Passage	Le train quitte la zone	Ouverture Barrière	- Arrêt du signal sonore. - Début de l'ouverture de la barrière.
Ouverture Barrière	- Arrêt du signal sonore. - Début de l'ouverture de la barrière.	Mode automatique (L'état initial du mode automatique)	- Feux verts pour véhicules. - Message "RAS".
Obstacle Détecté	Un obstacle est détecté	Obstacle Détecté	- Feu rouge clignotant. - Alarme sonore. - Arrêt du train.
Obstacle Détecté	L'obstacle est dégagé	Fermeture Barrière	- Barrière fermée. - Signal sonore pour passage du train.
Mode automatique	Le code PIN correct est entré	Idle (mode manuel)	- Retour au mode manuel.

- **Description des états et transitions**

États principaux de la FSM

1. **État Initial (Idle)**
    - La barrière est ouverte, les feux sont verts pour les véhicules et rouge pour le train, le message « RAS » affiché aux véhicules et piétons.
    - Le système est en attente de code pin pour basculer en mode automatique.
2. **État "mode automatique"**
    - La barrière est ouverte, les feux sont verts pour les véhicules et rouge pour le train, le message « RAS » affiché aux véhicules et piétons.
    - Le système est en attente de détection d'un train.
3. **État "Train Approchant"**

- Lorsqu'un train est détecté par les capteurs de présence, le système active les signaux pour avertir les véhicules et piétons.
- Passage du feu de signalisation au rouge pour les véhicules, au vert pour le train et affichage du message d'avertissement « STOP » affiché aux véhicules et piétons.

4. **État "Fermeture de la Barrière"**
    - La barrière commence à se fermer lorsque le train est suffisamment proche, et le feu rouge pour les véhicules reste allumé.
5. **État "Train en Passage"**
    - La barrière est fermée, et un signal sonore est émis pendant le passage du train.
    - Cet état persiste jusqu'à ce que le train ait complètement traversé le passage à niveau.
6. **État "Obstacle Détecté"**
    - Si un obstacle est détecté sur les voies (piéton ou véhicule), le système passe en alerte.
    - Le feu rouge pour le conducteur du train clignote et une alarme sonore est émise.
    - Le train s'arrête jusqu'à ce que l'obstacle quitte le passage.
7. **État "Ouverture de la Barrière"**
    - Une fois le train passé et les voies dégagées, la barrière s'ouvre.
    - Le feu pour les véhicules repasse au vert, et le système revient à l'état initial (mode automatique).

Transitions de la FSM

- **Idle (mode manuel) → mode automatique**: Transition lorsque le correct code pin est entrée.
- **Mode automatique → Train Approchant** : Transition lorsque le capteur détecte un train en approche.
- **Train Approchant → Fermeture de la Barrière** : Transition lorsque le train est à une distance où la fermeture doit être amorcée.
- **Fermeture de la Barrière → Train en Passage** : Transition une fois la barrière complètement fermée.

- **Train en Passage → Ouverture de la Barrière** : Transition lorsque le train a quitté la zone de passage.
- **Train Approchant → Obstacle Détecté** : Cette transition peut se produire lorsque le train approche et le capteur d'obstacle détecte un obstacle, déclenchant ainsi l'arrêt d'urgence et l'alarme.
- **Ouverture de la Barrière → mode automatique** : Transition lorsque la barrière est complètement ouverte et les feux pour les véhicules sont passés au vert.

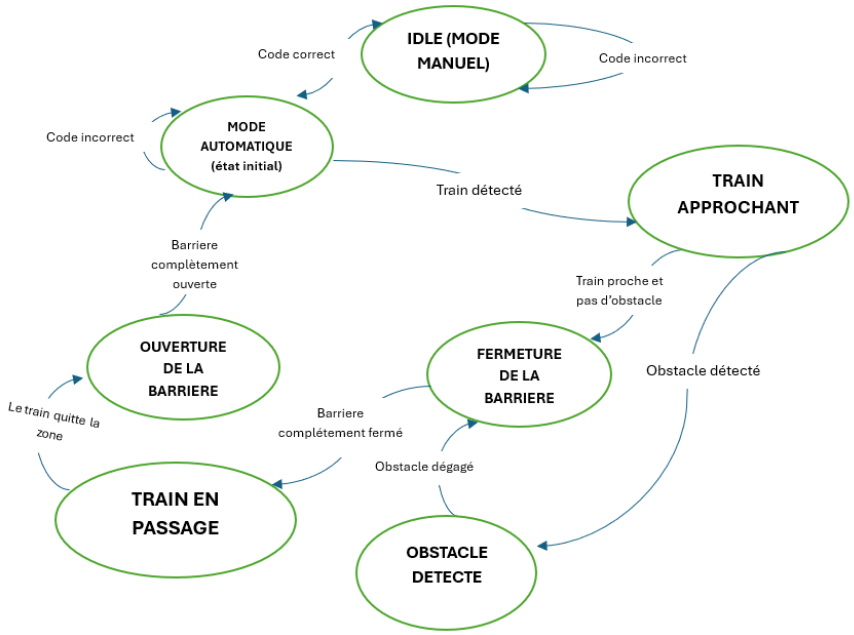


Figure 9: schéma FSM

VI. Flowchart et algorithme

Si on note :

S0 : état initial (mode manuel)

S1 : Mode automatique (état initial du mode automatique)

S2 : Train Approchant

S3 : Fermeture Barrière

S4 : Train en Passage

S5 : Obstacle Détecté

S6 : Ouverture Barrière

On va donner des valeurs à nos entrées pour faciliter la compréhension sur flowchart.

P_I: Pin_Input

C_P: Correct_Pin

C1 : Capteur_1 (prend la valeur 1 logique si le capteur à l'arrivée détecte la présence du train, sinon 0)

C2 : Capteur_2 (prend la valeur 1 logique si le capteur à la sortie détecte la présence du train, sinon 0)

C3 : Capteur_3 (prend la valeur 1 logique si le capteur d'obstacle détecte un obstacle, sinon 0)

B : état de la barrière (prend la valeur 1 logique si la barrière est fermée, sinon 0)

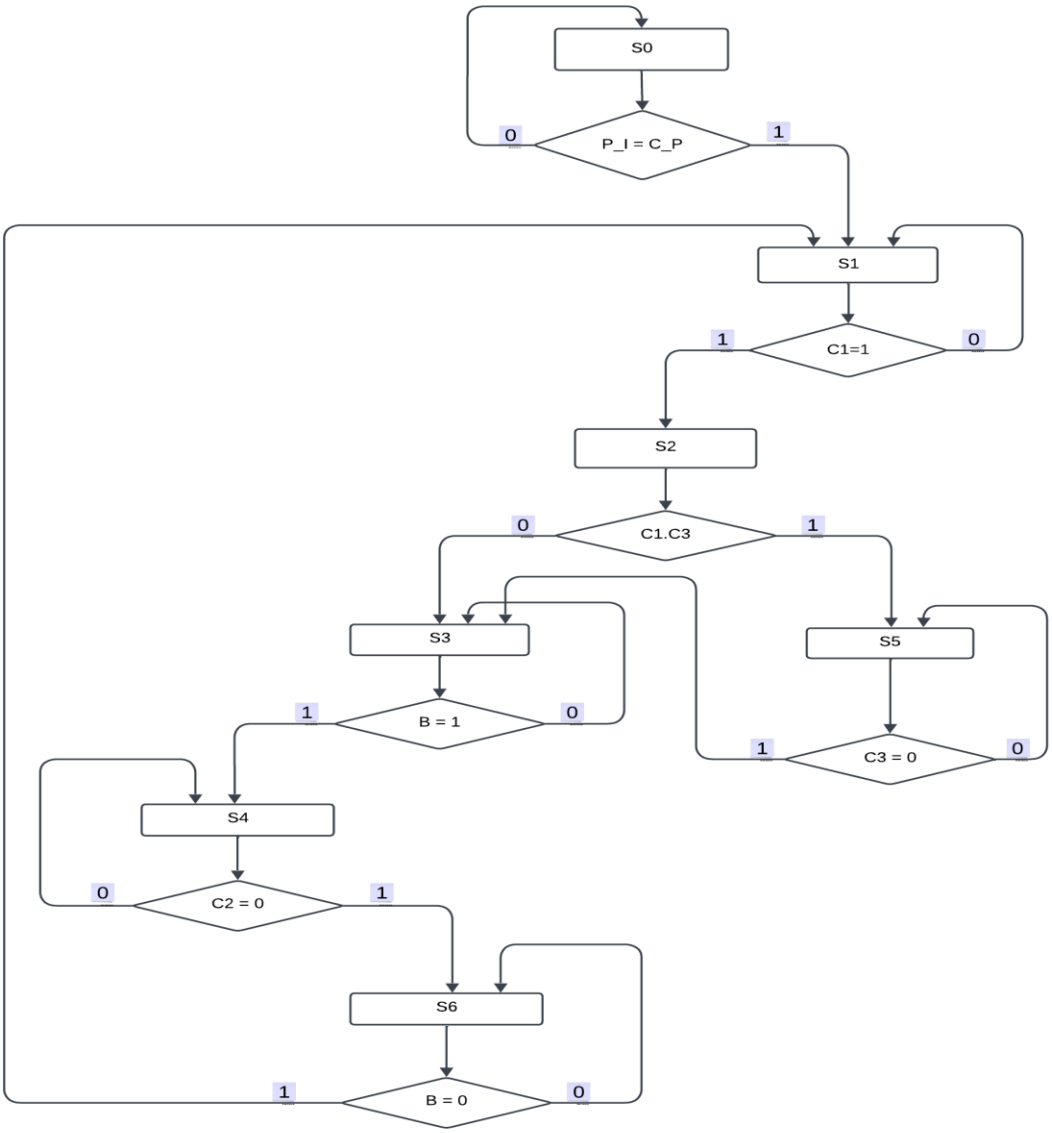


Figure 10: flowchart

VII. Développement et validation

a. Code VHDL

❖ *Changement de Mode (automatique et manuel)*

Ce module permet à l'utilisateur de basculer entre les modes automatique et manuel du système. La transition est déclenchée par la saisie d'un code PIN correct, avec une logique qui désactive ou active le système en conséquence.

Description du Processus VHDL

Le processus utilise deux signaux d'entrée principaux :

- **Pin_validator** : signal déclenchant la vérification du code PIN (transition descendante).
- **Reset** : signal pour réinitialiser l'état du système.

Le signal **pin_input** est comparé à une valeur préconfigurée, **correct_pin**, pour valider la saisie. Si le code PIN est correct :

- En cas de réinitialisation (**reset = '1'**), le système est désactivé (**system_active** à 0).
- Sinon, le mode du système est inversé grâce à **system_active <= not system_active**, permettant de basculer entre actif et inactif.

Code VHDL : Changement de Mode

```
process(pin_validator, reset)
begin
    if falling_edge(pin_validator) then
        -- Vérifier si le PIN entré est correct
        if pin_input = correct_pin then
            if reset = '1' then
                system_active <= '0'; -- Désactiver le système
            else
                -- Basculer l'état du système
                system_active <= not system_active; -- Activer/Désactiver selon l'état actuel
            end if;
        end if;
    end if;
end process;
```

❖ *Module de Gestion des Feux et de l'Affichage*

Ce module contrôle les signaux lumineux et l'affichage sur les segments hexadécimaux (HEX) pour indiquer les différents états du passage à niveau. Il gère les feux pour la route et la voie ferrée, tout en assurant une signalisation claire pour les conducteurs et les piétons en fonction des conditions détectées.

Description du Processus VHDL

Le processus utilise plusieurs signaux pour décider de l'état des feux et de l'affichage :

- **Reset** : réinitialise le système.
- **Key, key2, key3** : entrées indiquant la détection d'un obstacle ou la présence d'un train.
- **Buzzer** : signal sonore activé dans des conditions spécifiques.
- **Led0 à led3** : contrôlent les feux lumineux (route et voie ferrée).
- **HEX0 à HEX3** : affichent des messages ("STOP" ou "RAS") sur les segments LED.

Le fonctionnement est divisé en trois cas principaux :

1. **Réinitialisation (reset = '1')** :
    - Tous les feux sont éteints.
    - Les segments hexadécimaux affichent des segments éteints (**"1111111"**).
2. **Condition critique (obstacle détecté + train)** :
    - Les feux de la route principale passent au rouge.
    - Un feu clignotant est activé sur la voie ferrée.
    - Le mot **"STOP"** est affiché sur les segments hexadécimaux.
    - Le buzzer est activé en mode **buzzer_state1**.
3. **Présence de train uniquement** :
    - Les feux de la route principale passent au rouge.
    - Les feux de la voie ferrée passent au vert.
    - Le mot **"STOP"** est affiché.
    - Le buzzer est activé en mode **buzzer_state**.
4. **État normal (RAS)** :
    - Les feux de la route principale sont verts, et ceux de la voie ferrée restent rouges.
    - Le mot **"RAS"** est affiché sur les segments hexadécimaux.

Code VHDL : Gestion des Feux et de l'Affichage

```
if reset = '1' then
    -- Réinitialisation
    led0 <= '0'; -- Feu vert route principale
    led1 <= '0'; -- Feu rouge route principale
    led2 <= '0'; -- Feu rouge voie de fer
    led3 <= '0'; -- Feu vert voie de fer
    HEX3 <= "1111111"; -- Éteindre tous les segments de HEX3
    HEX2 <= "1111111"; -- Éteindre tous les segments de HEX2
    HEX1 <= "1111111"; -- Éteindre tous les segments de HEX1
    HEX0 <= "1111111"; -- Éteindre tous les segments de HEX0
else
    if (key = '0' and key2 = '0') or (key3 = '0' and key2 = '0') then
        -- Situation critique : STOP
        led0 <= '0'; -- Feu vert route principale => rouge
        led1 <= '1'; -- Feu rouge route principale
        led2 <= led_state; -- Feu rouge voie de fer (clignotant)
        led3 <= '0'; -- Feu vert voie de fer
        HEX3 <= LETTER_S; HEX2 <= LETTER_T; HEX1 <= LETTER_O; HEX0 <= LETTER_P;
        buzzer <= buzzer_state1; -- Buzzer activé
    elsif train = '1' then
        -- Présence de train
        led0 <= '0'; led1 <= '1'; led2 <= '0'; led3 <= '1';
        HEX3 <= LETTER_S; HEX2 <= LETTER_T; HEX1 <= LETTER_O; HEX0 <= LETTER_P;
        buzzer <= buzzer_state; -- Buzzer activé
    else
        -- État normal : RAS
        led0 <= '1'; led1 <= '0'; led2 <= '1'; led3 <= '0';
        HEX3 <= LETTER_R; HEX2 <= LETTER_A; HEX1 <= LETTER_S; HEX0 <= LETTER_L;
    end if;
end if;
```

Figure 11: code affichage, leds, messages et buzzer

Explication des Éléments Affichés

- **"STOP"** : Indiqué lorsque les conditions critiques sont détectées, alertant d'un danger immédiat.
- **"RAS"** : Signale que tout est normal et qu'il n'y a aucune anomalie.

❖ *Gestion des Feux Clignotants*

Ce module contrôle l'état clignotant d'un feu en fonction de conditions spécifiques, telles que la détection simultanée d'un obstacle et d'un train. Il utilise un compteur pour alterner l'état du feu à un intervalle déterminé.

Description du Processus VHDL

Le processus repose sur une horloge (**clk**) et des signaux d'entrée (**key, key2, key3**) pour déclencher l'activation du feu clignotant.

Code VHDL : Gestion des Feux Clignotants

```
process(clk, key, key2)
begin
    if rising_edge(clk) then
        -- Vérification des conditions critiques
        if (key = '0' and key2 = '0') or (key3 = '0' and key2 = '0') then
            -- Gestion du compteur
            if (counter >= 50000000) then
                counter <= 0; -- Réinitialisation du compteur
                led_state <= not led_state; -- Inversion de l'état du feu
            else
                counter <= counter + 1; -- Incrémentation du compteur
            end if;
        end if;
    end if;
end process;
```

Figure 12: code feux clignotants

❖ *Module de Détection Ultrasonique*

Ce module contrôle les signaux TRIG et ECHO pour mesurer la distance entre le capteur ultrasonique et un objet. En détectant si la distance est inférieure à un seuil donné, il active ou désactive un signal de détection (**détecte**) pour chaque capteur.

Description des Étapes Principales

1. **Génération de l'Impulsion TRIG** :
    - Le signal **trig** est maintenu à 1 pendant une courte durée (500 cycles d'horloge, soit environ 10 µs avec une horloge de 50 MHz).
    - Après cette impulsion, il est réinitialisé à 0.
    - Cette séquence se répète périodiquement (toutes les 250 ms) pour chaque capteur.
2. **Gestion du Signal ECHO** :
    - Lorsque le signal **echo** passe à 1, un compteur commence à s'incrémenter pour mesurer la durée de l'impulsion.
    - Lorsque **echo** revient à 0, la durée mesurée est utilisée pour calculer la distance de l'objet détecté.
3. **Décision Basée sur la Distance** :
    - Si la durée mesurée est inférieure à un seuil (par exemple, 40 000 cycles), l'objet est considéré comme proche et le signal **détecte** est activé (1).
    - Sinon, le signal **détecte** est désactivé (0).

Code VHDL Général pour Capteurs

```
process(clk)
begin
    if rising_edge(clk) then
        for i in 0 to N-1 loop  -- N : Nombre de capteurs
            -- Générer l'impulsion TRIG
            if trig_counter(i) = 0 then
                trig(i) <= '1';  -- TRIG à '1' pour le capteur i
            elsif trig_counter(i) = 500 then
                trig(i) <= '0';  -- TRIG à '0' après 10 µs
            end if;

            -- Incrémenter le compteur de TRIG
            if trig_counter(i) < 12_500_000 then  -- Répéter toutes les 250 ms
                trig_counter(i) <= trig_counter(i) + 1;
            else
                trig_counter(i) <= 0;  -- Réinitialiser le compteur TRIG
            end if;

            -- Gestion du signal ECHO
            if echo(i) = '1' then
                if echo_active(i) = '0' then
                    echo_counter(i) <= 0;  -- Réinitialiser le compteur ECHO
                    echo_active(i) <= '1';  -- Début de l'impulsion ECHO
                else
                    echo_counter(i) <= echo_counter(i) + 1;  -- Incrémenter le compteur
                end if;
            elsif echo(i) = '0' and echo_active(i) = '1' then
                echo_active(i) <= '0';  -- Fin de l'impulsion ECHO

                -- Calcul de la distance et activation/désactivation de DETECTE
                if echo_counter(i) < 40000 then
                    detecte(i) <= '1';  -- Objet proche
                else
                    detecte(i) <= '0';  -- Objet éloigné
                end if;
            end if;
        end loop;
    end if;
end process;
```

Figure 13: code capteurs ultrasons

❖ *Module de Contrôle du Servomoteur*

Le module gère la largeur d'impulsion (Pulse Width Modulation, PWM) pour contrôler l'angle de rotation du servomoteur. Il permet d'ouvrir ou de fermer les barrières en ajustant la largeur de l'impulsion en fonction de l'état d'un commutateur (switch).

Description du Fonctionnement

1. **Ouverture de la Barrière** :
    - Si le commutateur est activé (**switch = '1'**), la largeur d'impulsion augmente progressivement par incréments (par exemple, de **10** unités) jusqu'à atteindre une valeur maximale prédéfinie (**MAX_PULSE_WIDTH**), correspondant à une ouverture de 90°.
2. **Fermeture de la Barrière** :

- Si le commutateur est désactivé (**switch = '0'**), la largeur d'impulsion diminue progressivement par décrémentation jusqu'à atteindre une valeur minimale prédéfinie (**MIN_PULSE_WIDTH**), correspondant à une position fermée (0°).

3. **Sécurisation des Limites** :
    - La largeur d'impulsion est maintenue dans des limites sécurisées pour éviter tout comportement inattendu du servomoteur.

Code VHDL pour le Servomoteur

```
process(clk)
begin
    if rising_edge(clk) then
        -- Si le switch est activé, augmenter la largeur d'impulsion pour ouvrir la barrière
        if switch = '1' then
            if pulse_width < MAX_PULSE_WIDTH then
                pulse_width <= pulse_width + 10;  -- Incrémenter la largeur d'impulsion
            end if;
        else
            -- Si le switch est désactivé, réduire la largeur d'impulsion pour fermer la barrière
            if pulse_width > MIN_PULSE_WIDTH then
                pulse_width <= pulse_width - 10;  -- Réduire la largeur d'impulsion
            end if;
        end if;
    end if;
end process;
```

Figure 14: code servomoteur

Explication du Signal PWM

Le servomoteur utilise un signal PWM pour positionner la barrière. La durée de l'impulsion (par exemple, entre 1 ms et 2 ms dans une période de 20 ms) détermine l'angle de rotation du servomoteur :

- Une largeur minimale d'impulsion (**MIN_PULSE_WIDTH**) correspond à 0°.
- Une largeur maximale d'impulsion (**MAX_PULSE_WIDTH**) correspond à 90°.

❖ *Module de Contrôle du Buzzer*

Le module gère l'émission sonore du buzzer en alternant entre une période active (**1 seconde ON**) et une période inactive (**0,5 seconde OFF**). Lorsque le buzzer est actif, il utilise un signal PWM pour générer une tonalité sonore.

Description du Fonctionnement

1. **Génération des Périodes ON/OFF** :
    - Un compteur (**delay_counter**) contrôle l'activation et la désactivation du buzzer.
    - Lorsque **delay_counter** atteint une valeur définie (**ONE_SEC** pour 1 seconde), il bascule l'état de **buzzer_enable**, permettant ainsi d'activer ou désactiver le buzzer de manière périodique.
2. **Activation avec le Signal PWM** :

- Si **buzzer_enable** est actif (**'1'**), le signal PWM (**pwm_signal**) est appliqué au buzzer pour émettre un son.
- Si **buzzer_enable** est inactif (**'0'**), la sortie du buzzer est désactivée (**'0'**).

b. Schémas électriques des connexions

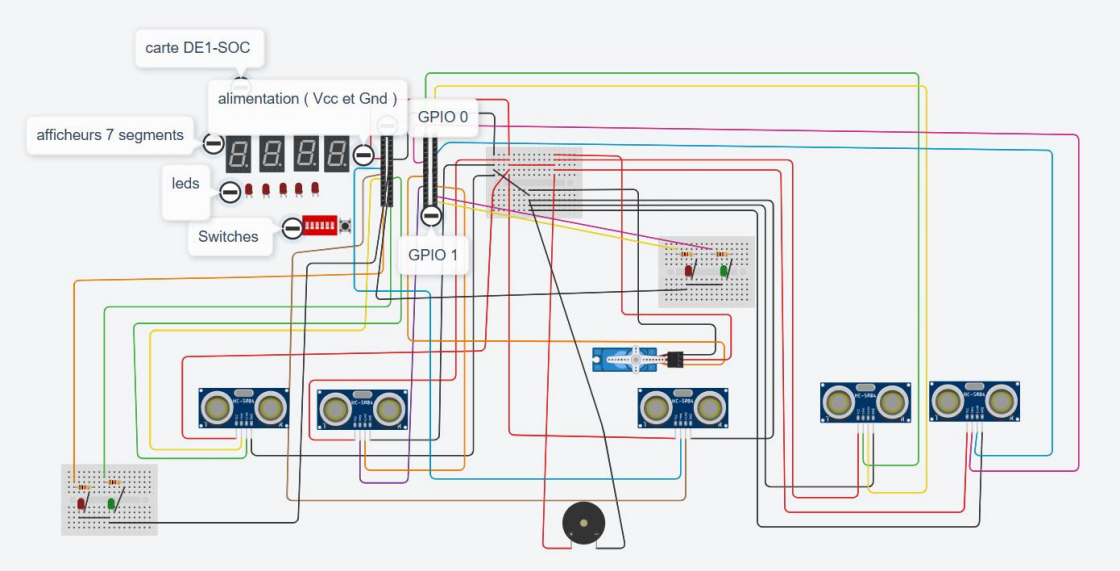


Figure 15: schémas électriques des connexions

Le schéma électrique présenté ci-dessous illustre les connexions entre les différents composants du système de passage à niveau automatisé. Chaque module, tel que le capteur ultrasonique, le servomoteur, les LEDs de signalisation, et les interrupteurs, est interconnecté à la carte DE1-SoC, qui joue le rôle de contrôleur principal. Les broches de la carte FPGA sont assignées de manière stratégique pour assurer une communication efficace et respecter les contraintes de conception. Ce schéma permet de visualiser clairement les relations entre les entrées (capteur, interrupteurs) et les sorties (LEDs, servomoteur) ainsi que l'alimentation des différents éléments du circuit. Il constitue une référence essentielle pour le câblage et le débogage du système.

c. Simulations fonctionnelles

-Servomoteurs : La simulation a permis de tester les mouvements du servomoteur en mode automatique, contrôlé par une logique PWM. L'objectif était de s'assurer que le servomoteur bascule correctement entre ses positions : 0° (barrière fermée) et 90° (barrière ouverte), selon les signaux du capteur.

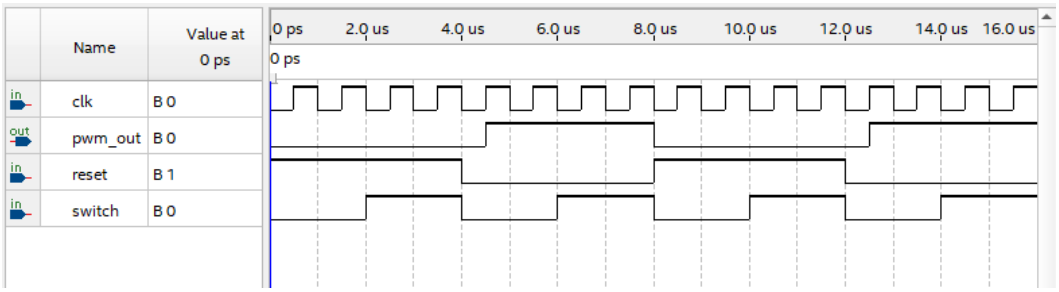


Figure 16: simulation servomoteurs

- buzzer : Le buzzer, utilisé comme signal d'alerte sonore, a été simulé pour s'activer lorsque le capteur détecte un train ou lorsqu'un signal de danger est émis. La simulation a vérifié que la fréquence et la durée des signaux sonores correspondaient aux conditions définies dans le code VHDL.

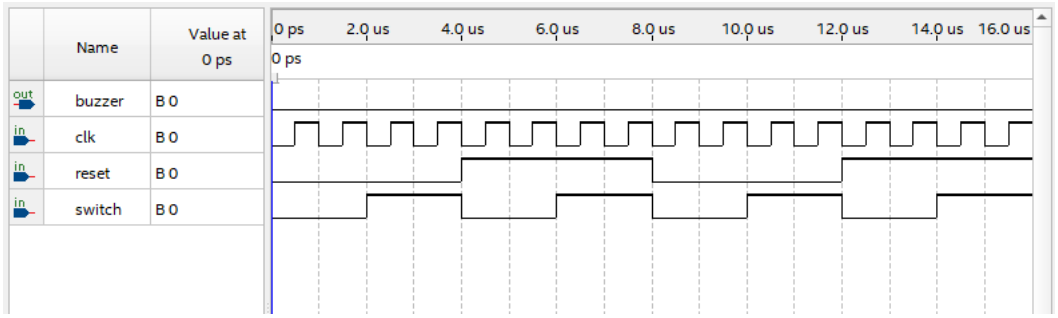


Figure 17: simulation buzzer

- **Messages** : Les messages textuels simulés, "STOP" et "RAS" (Rien À Signaler), ont été affichés en fonction de l'état du système. "STOP" apparaît lorsqu'un train est détecté ou lorsque la barrière est fermée, tandis que "RAS" s'affiche lorsque la barrière est ouverte et qu'aucun danger n'est détecté. Ces messages permettent de communiquer l'état du passage à niveau de manière claire et précise.

- **Feux** : Les LEDs de signalisation ont été simulées pour représenter visuellement l'état du passage à niveau. Lorsque la barrière est fermée ou en cours de fermeture, la LED rouge s'allume pour indiquer un danger imminent. La LED verte s'allume uniquement lorsque la barrière est complètement ouverte et qu'il n'y a aucun obstacle détecté.

- **Code PIN** : Le basculement entre le mode automatique et le mode manuel a été simulé en introduisant le code PIN "1010 ".

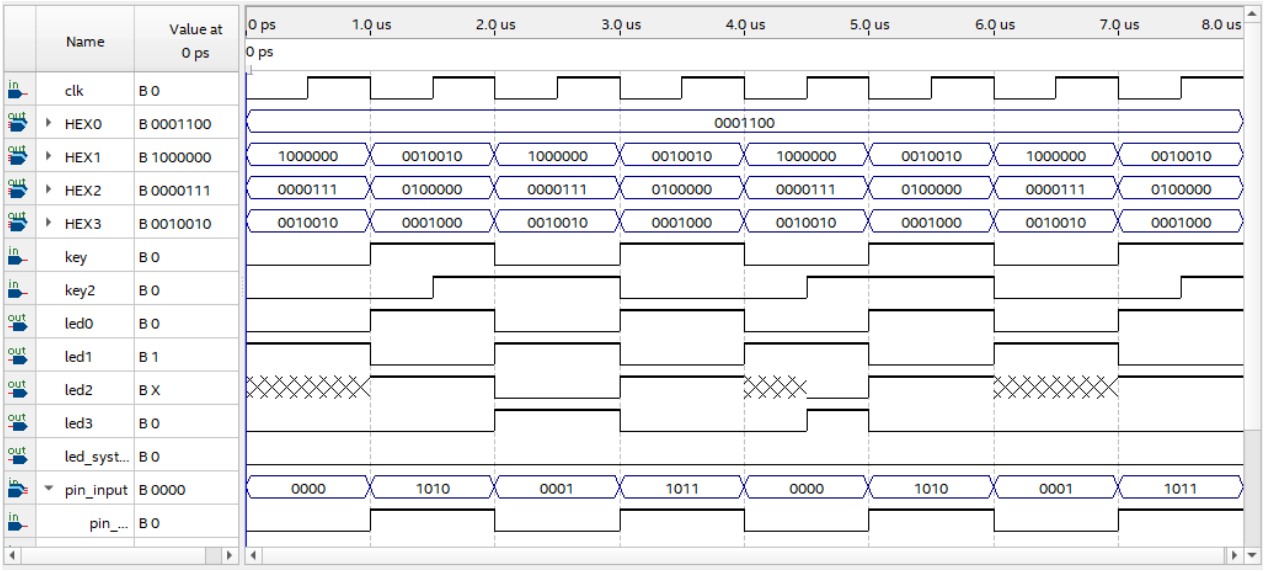


Figure 18: simulation messages, leds et code pins

d. Représentations Graphiques du Système

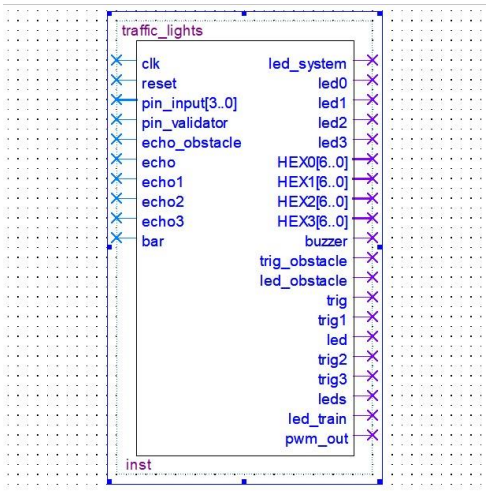


Figure 19: Symbole graphique du système

Le symbole graphique du système présenté ci-dessous représente une abstraction visuelle de l'ensemble des fonctionnalités implémentées dans le projet. Ce symbole regroupe les entrées principales, telles que les signaux des capteurs et les commandes utilisateur, ainsi que les sorties, notamment les signaux de contrôle des LEDs, du servomoteur et de l'alarme. Il simplifie la compréhension du système en mettant en évidence les interfaces externes tout en encapsulant la complexité des modules internes. Ce diagramme est particulièrement utile pour l'intégration

dans des conceptions plus larges ou pour la documentation du système, car il fournit une vue claire et succincte de son fonctionnement.

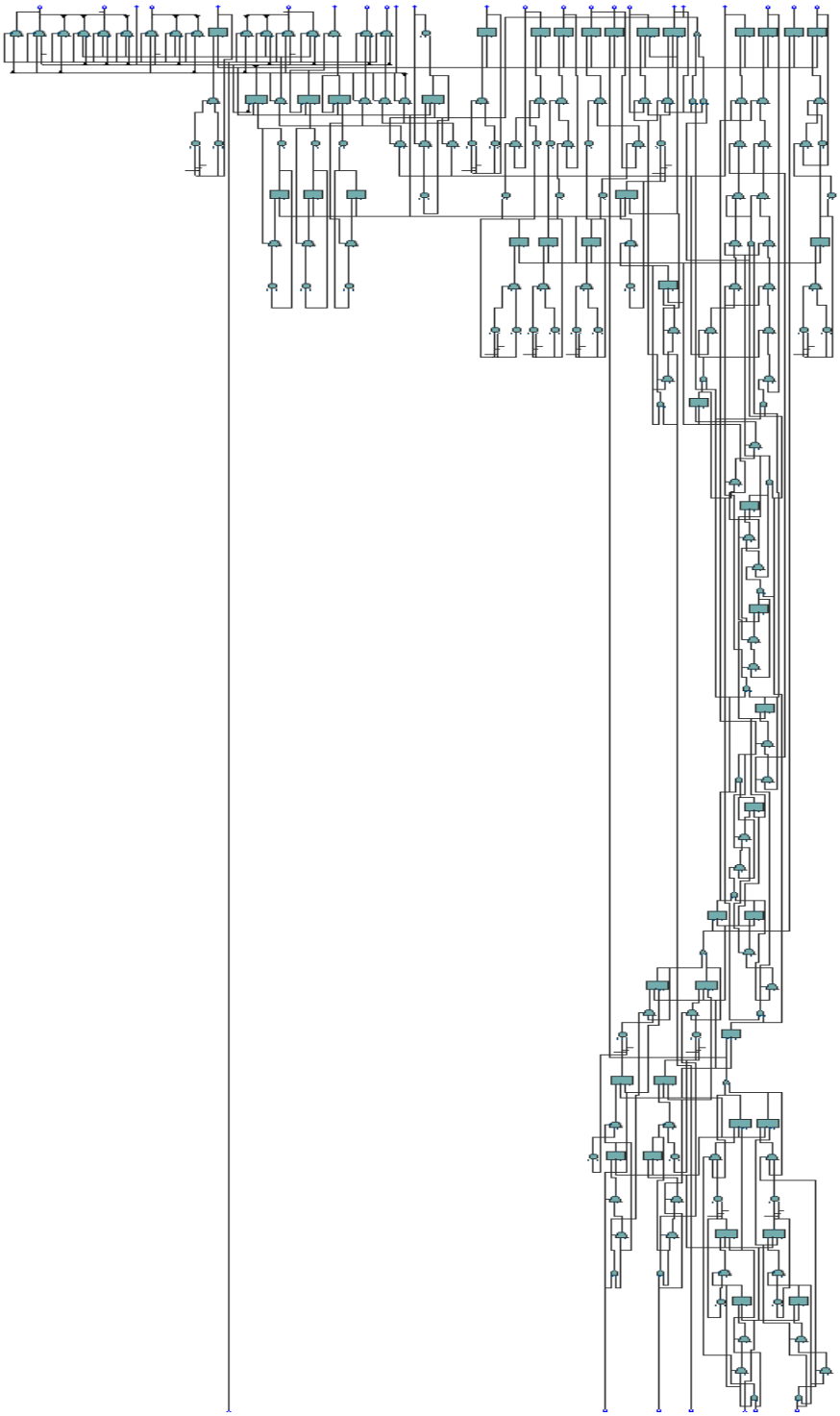


Figure 20: Netlist du système

Visualisation de la Netlist du Système

Le Netlist Viewer présenté ci-dessous offre une vue détaillée des connexions logiques générées après la synthèse du code VHDL. Il permet de visualiser la hiérarchie des modules du système, incluant les composants principaux tels que le module de détection, le contrôle du servomoteur, et la gestion des signaux de signalisation.

Chaque élément est représenté avec ses interconnexions, mettant en évidence le flux de données entre les différents blocs. Cette représentation graphique est essentielle pour valider l'implémentation des circuits logiques, identifier les éventuelles erreurs, et optimiser les performances globales du système avant la phase de mise en œuvre matérielle.

VIII. Difficultés rencontrées lors de la conception

Lors de la conception du système, plusieurs défis ont été rencontrés. Ces difficultés concernaient principalement des aspects méthodologiques, organisationnels et techniques et ont nécessité des efforts supplémentaires. Voici quelques difficultés rencontrées :

- **Intégration du module GSM** : Difficulté de comprendre son fonctionnement interne, ses protocoles de communication (AT commands) et de l'implémenter sur la carte FPGA.

- **Développement et validation du code VHDL** : Complexité de diviser le code en modules fonctionnels tout en assurant leur intégration cohérente. En effet, il fallait s'assurer que chaque module fonctionne indépendamment avant leur intégration; l'intégration cohérente des différents modules a donc nécessité des simulations fonctionnelles et temporelles approfondies, pour identifier et résoudre les erreurs, afin de prévenir tout comportement inattendu du système.

- **Contrôle du servomoteur en mode manuel** : Difficulté à intégrer une commande manuelle du servomoteur, tout en maintenant la logique PWM fonctionnelle en mode automatique. En effet, nous voulions être capable de contrôler l'ouverture et la fermeture de la barrière tant en mode automatique qu'en mode manuel.

- **Synchronisation du signal PWM** : Complexité à synchroniser le signal PWM avec des signaux de commandes spécifiques pour éviter des mouvements erratiques du servomoteur. La difficulté résidait dans la nécessité de maintenir une fréquence stable et des largeurs d'impulsions précises, tout en prenant en compte les signaux de commande provenant des capteurs.

- **Conception d'un chemin à double sens :** Difficulté à gérer les signaux pour concevoir la logique d'un chemin à double sens.

Ces défis sont partie intégrante du processus de développement, car ils témoignent des multiples facettes techniques du projet et permettent d'affiner les compétences, d'optimiser les solutions, et d'améliorer la robustesse du système final. Finalement, tous ces défis ont été surmontés excepté l'intégration du module GSM dont le code des différents essais est en annexe.

Si nous avions disposé de plus de temps, nous aurions certainement pu apporter des améliorations à notre système, notamment utiliser une FSM dans la programmation (coder les différents modules, et les assembler dans un code principal) afin d'optimiser l'aspect temporel du système.

IX. Conclusion et perspectives

❖ *Conclusion*

Le développement d'un système de passage à niveau automatisé basé sur FPGA a permis de répondre efficacement aux exigences de sécurité et de fiabilité dans un environnement critique. En intégrant des technologies modernes telles que les capteurs ultrasoniques, les servomoteurs, et un contrôle numérique par FPGA, ce projet a démontré une solution capable de détecter des obstacles, gérer les signaux lumineux, et contrôler l'ouverture et la fermeture des barrières en temps réel.

Le choix de la carte DE1-SoC a offert une flexibilité accrue grâce à l'utilisation combinée du matériel programmable et des outils de simulation. Les modules développés (gestion des capteurs, contrôle des feux et barrières, changement de mode automatique/manuel, etc.) ont été validés par des tests fonctionnels et temporels, garantissant une synchronisation précise entre les différentes parties du système.

Ce projet illustre l'importance de l'approche modulaire et des méthodologies de validation rigoureuses dans la conception de systèmes embarqués critiques, où la sécurité est prioritaire.

❖ *Perspectives*

Bien que le système développé réponde aux objectifs initiaux, plusieurs pistes d'amélioration peuvent être envisagées pour renforcer ses performances et étendre son champ d'application :

1. **Ajout de fonctionnalités avancées** :
    a. Ajouter une connexion IoT pour surveiller et contrôler le système à distance.

b. Inclure un système de diagnostic en temps réel pour détecter les pannes (mécaniques ou électroniques) et alerter les opérateurs.

2. **Optimisation du matériel** :

   a. Réduire la consommation énergétique du système en utilisant des techniques d'optimisation matérielle.

   b. Explorer d'autres plateformes FPGA offrant une plus grande intégration ou des ressources spécifiques adaptées aux systèmes embarqués. (Explorer les FPGA dotés de modèles d'intelligence artificielle).

3. **Sécurité renforcée** :

   a. Ajouter un support multi-langue pour s'assurer que tous les usagers du croisement ferroviaire comprennent les messages d'alertes.

   b. Implémenter des algorithmes de cryptographie pour sécuriser les communications entre les différents modules, notamment dans le cas de l'utilisation d'un mode Open RAN ou d'un réseau IoT.

   c. Ajouter des capteurs supplémentaires pour améliorer la fiabilité, comme des caméras ou des radars pour détecter les objets dans des environnements difficiles (brouillard, pluie, etc.).

En conclusion, ce projet constitue une base solide pour le développement de systèmes intelligents basés sur FPGA dans des environnements nécessitant à la fois la sécurité, la flexibilité, et la réactivité (en temps réel). Les évolutions envisagées ouvrent des perspectives prometteuses, tant dans le domaine des transports que dans d'autres secteurs industriels.

X. Bibliographie

- Cours GEN 1243, chapitre II « Flot de conception des systèmes basés sur les composants logiques programmables », Parties 1 et 2, par Mr. Rahmani, Naim Mohamed.
- Cours GEN 1243, chapitre V « Machine d'états algorithmiques (ASM) », Partie 1, par Mr. Rahmani, Naim Mohamed.
- *FPGA Projects*. (s. d.). FPGA4student.com. https://www.fpga4student.com/p/fpga-projects.html
- Cuvelier, T. (s. d.). *Introduction au langage VHDL*. Developpez.com. https://tcuvelier.developpez.com/tutoriels/vhdl/introduction-langage/
- https://sist.sathyabama.ac.in/sist_naac/documents/1.3.4/b.e-ece-batchno-12.pdf
- (PDF) Automatic Railway Level Crossing Using LoRa
- *Automated Railway Crossing System: A Secure and Resilient Approach*. (2023, 12 October). IEEE Conference Publication | IEEE Xplore. https://ieeexplore.ieee.org/document/10315973

XI. Annexes



SIM7600A pour le GSM

Code pour le module GSM

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity gsm is
    port (
        clk         : in  std_logic;   -- Horloge principale
        reset       : in  std_logic;   -- Reset
        start_tx    : in  std_logic;   -- Signal pour commencer l'envoi
        uart_tx     : out std_logic;   -- Ligne de transmission UART
        busy        : out std_logic    -- Indique que l'UART est occupé
    );
end gsm;

architecture Behavioral of gsm is
    constant CLK_FREQ : integer := 50000000; -- Fréquence de l'horloge (50 MHz)
    constant BAUD     : integer := 115200;   -- Baud rate (115200 pour SIM7600A)
    constant BIT_TIME : integer := CLK_FREQ / BAUD; -- Temps d'un bit

    type state_type is (IDLE, START_BIT, DATA_BITS, STOP_BIT);
    signal state      : state_type := IDLE;

    signal bit_counter : integer range 0 to 9 := 0; -- Compteur de bits
    signal char_index  : integer := 0; -- Index du caractère à envoyer
    signal shift_reg   : std_logic_vector(7 downto 0); -- Registre de décalage pour le caractère
    signal clk_counter : integer := 0;

    -- Commandes AT et message à envoyer
    constant CMD_AT : string := "AT+CMGF=1" & character'val(0);  -- Mode texte
    constant CMD_SEND_SMS : string := "AT+CMGS="" +15148949097 """ & character'val(0);  -- Numéro + SMS
    constant MESSAGE_CONTENT : string := "ALERT: Anomaly detected" & character'val(0);  -- Contenu du message

    constant MESSAGE_LENGTH : integer := MESSAGE_CONTENT'length;

begin

    process(clk, reset)
    begin
        if reset = '1' then
            state       <= IDLE;
            uart_tx     <= '1'; -- Ligne UART inactive
            busy        <= '0';
            bit_counter <= 0;
            clk_counter <= 0;
            char_index  <= 0;
        elsif rising_edge(clk) then
            case state is
                when IDLE =>
                    uart_tx <= '1'; -- Ligne UART inactive
```

```
                    busy <= '1';
                    clk_counter <= 0;
                    bit_counter <= 0;
                    char_index <= 0;  -- Commence par la commande AT
                end if;

            when START_BIT =>
                uart_tx <= '0'; -- Start bit
                if clk_counter = BIT_TIME - 1 then
                    clk_counter <= 0;
                    state <= DATA_BITS;
                else
                    clk_counter <= clk_counter + 1;
                end if;

            when DATA_BITS =>
                uart_tx <= shift_reg(0); -- Envoi du bit de données
                if clk_counter = BIT_TIME - 1 then
                    clk_counter <= 0;
                    shift_reg <= '0' & shift_reg(7 downto 1); -- Décalage à droite
                    if bit_counter = 7 then
                        bit_counter <= 0;
                        if char_index < MESSAGE_LENGTH then
                            -- Passer à l'index suivant du message
                            char_index <= char_index + 1;
                            state <= START_BIT;
                        else
                            -- Si le message est envoyé, revenir à IDLE
                            state <= STOP_BIT;
                        end if;
                    else
                        bit_counter <= bit_counter + 1;
                    end if;
                else
                    clk_counter <= clk_counter + 1;
                end if;

            when STOP_BIT =>
                uart_tx <= '1'; -- Stop bit
                if clk_counter = BIT_TIME - 1 then
                    clk_counter <= 0;
                    if char_index < MESSAGE_LENGTH then
                        state <= START_BIT;
                    else
                        state <= IDLE;
                        busy <= '0';
                    end if;
                else
                    clk_counter <= clk_counter + 1;
                end if;
        end case;
    end if;
  end process;
end Behavioral;
```